# INF2006 Team Project Report
# Roomly

Campus Room Booking Service

**GROUP: Group_G11**


| Name | Student ID | Student Email |
|---|---|---|
| SHERMAINE CHAN JING XUAN | 2501469 | 2501469@sit.singaporetech.edu.sg |
| SU YING STANBERLEY | 2501376 | 2501376@sit.singaporetech.edu.sg |
| TEO XUE LING SHARLENE | 2503489 | 2503489@sit.singaporetech.edu.sg |
| WEI YUCHEN | 2503041 | 2503041@sit.singaporetech.edu.sg |


**Submission Date: 11 October 2026**

# CONTENTS

**1. Problem, Users and Success Criteria..... 3**
1.1 Problem Statement.....3
1.2 Users.....3
1.3 Success Criteria.....3
1.4 Scope.....4
**2. Solution Overview and Architecture..... 4**
2.1 Overview.....4
2.2 Architecture Diagram.....4
2.3 API Surface.....5
2.4 Service and Deployment Model.....5
**3. Cloud Service/Deployment Choices and Trade-offs.....5**
3.1 Chosen Approach.....5
3.2 Alternatives Considered.....6
3.3 Key Assumptions and Constraints.....6
3.4 Cost-control Measures.....6
**4. Implementation, Data Design and Security Controls..... 6**
4.1 Application.....6
4.2 Booking Validation Rules.....7
4.3 Cancellation.....7
4.4 Data Model.....7
4.4.1 Attendance and Check-in.....7
4.5 Security Controls.....8
**5. Analytics or AI/ML Feature: Data, Method, Evaluation and Limitations..... 8**
5.1 Dataset and Provenance.....8
5.2 Method.....8
5.3 Evaluation.....8
5.4 Results and Outputs.....9
5.5 Limitations.....9
5.6 Responsible Use.....9
**6. Testing, Scalability/Resilience and Monitoring Results..... 9**
6.1 Test Summary.....9
6.1.1 Backend Test Summary..... 10
6.2 Scalability and Resilience Mechanism..... 10
6.3 Monitoring.....11
6.4 Known Limitations..... 11
**7. Cost, Sustainability and Operational Considerations..... 11**
**8. Team Contribution, Ethical Considerations and Reflection.....12**
8.1 Contribution Matrix..... 12
8.2 Ethical Considerations.....13
8.3 Individual Reflections..... 13
**References.....14**
**Appendices.....14**

Appendix A: Evidence Index..... 14

# 1. Problem, Users and Success Criteria

## 1.1 Problem Statement

Campus study rooms are limited and booking them is informal. Students coordinate over messages, rely on memory and may only discover a clash when they arrive. There is no single view of room availability, bookings or demand, causing double-bookings, wasted trips and limited data for room-capacity or scheduling decisions.

Roomly addresses this with a secure cloud-hosted booking service combining a web application, participant-aware booking workflow and historical reservation-demand analytics.

## 1.2 Users

| **User** | **Needs** | **What they can do in Roomly** |
|---|---|---|
| Student | Find an available room for an hour, book it with group members and can release the slot if plans change | Search rooms by date & group size, book a one hour slot, add participants, view their bookings, cancel bookings they organised, check in |
| Organiser (the student who books the room) | Keeps the group's booking reliable | Same features as a student. The organiser is included automatically and can cancel a future booking |
| Participant | To see if they were added into the booking and check in | View the shared booking and check in |
| Administrator | Keep the room list accurate and understand how rooms are reserved | Add and edit rooms (name, capacity, location, active status), view historical reservation analytics and demand patterns |

## 1.3 Success Criteria

| | **Criteria** | **How it is measured** |
|---|---|---|
| **S1** | A booking persists | After creating a booking and reloading the page, it still appeared in "My Bookings" with the same reference |
| **S2** | Participants see the same booking | Each listed participant sees the same booking reference when signed in with their own account |
| **S3** | Cancellation releases the slot | After the organiser cancels, the booking shows cancelled and the same slot can be booked again |

| | | |
|---|---|---|
| **S4** | Unauthorised users cannot manage rooms | A student sees no admin tab and a student's request to an admin room endpoint is rejected by the server |
| **S5** | Conflicting bookings are rejected | A request for an occupied slot is refused by the server |
| **S6** | Sign-in works in normal use | Student and admin can sign in, sign out, and sign in again with no repeated error |
| **S7** | Errors are understandable | Loading, empty and failed requests each show a message |

### 1.4 Scope

In scope: room booking, participant management, cancellation and reservation-demand analytics.

Out of scope: real campus room integration, payment and production identity federation.

# 2. Solution Overview and Architecture

### 2.1 Overview

Roomly uses a locally served HTML/CSS/JavaScript frontend with an AWS-hosted backend in us-east-1. Users authenticate through Amazon Cognito, authenticated requests go through API Gateway to Lambda, while DynamoDB stores users, rooms, bookings and reservation records. This gives users on different computers access to shared cloud data subject to account permissions.

Historical analytics are separate from live bookings: the notebook generates summary JSON and grouped CSV outputs, which are stored in a private S3 bucket and read by an analytics Lambda for the dashboard. CloudWatch provides execution and API logging.

## 2.2 Architecture Diagram

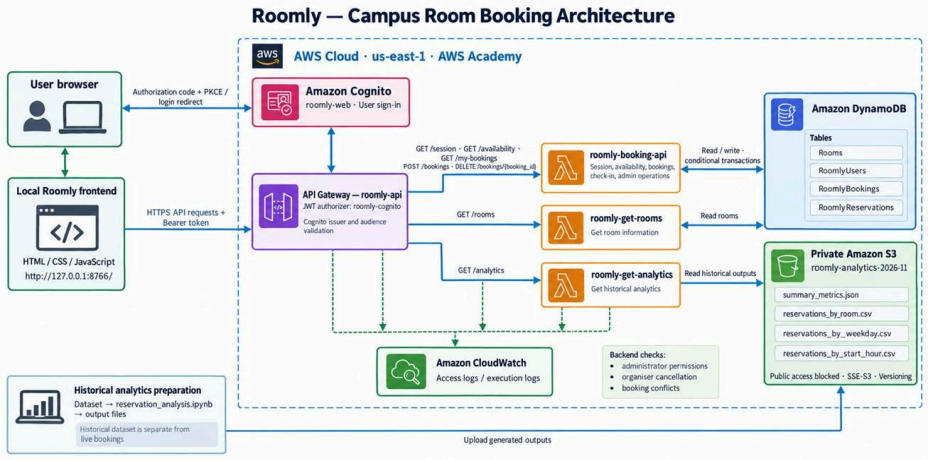



*Figure 1. Roomly architecture. The browser does not access DynamoDB or S3 directly, API Gateway and Lambda mediate requests, while Lambda execution roles and backend checks enforce access.*

Roomly follows a cloud-based architecture where the local Roomly frontend communicates with AWS services through API Gateway. Users access the frontend through their browser and authenticate using Amazon Cognito, which handles the sign-in process. After authentication, the frontend sends HTTPS requests with a Bearer token to API Gateway where the Cognito issuer and audience are validated before requests are passed to the appropriate backend services.

The main booking functions are handled by the roomly-booking-api Lambda, while roomly-get-rooms manages room information and roomly-get-analytics provides historical analytics. DynamoDB stores users, rooms, bookings and reservation records while S3 stores the generated analytics outputs. CloudWatch provides monitoring and logs while backend checks enforce admin permissions, cancellation authorisation and booking conflicts.

Overall data flow:
User → Roomly Frontend → Cognito/API Gateway → Lambda → DynamoDB/S3 with CloudWatch providing monitoring across the cloud components.

## 2.3 API Surface

The API supports session identification, room listing, participant selection, availability, booking creation/retrieval, cancellation, check-in and administrative room operations. Representative routes are GET /session, GET /availability, POST /bookings, GET /my-bookings, DELETE /bookings/{booking_id}, GET /admin/rooms and GET /analytics. Final route definitions and authorizers are documented in src/infrastructure/api-gateway-config.md.

## 2.4 Service and Deployment Model

Lambda, API Gateway, DynamoDB, Cognito and S3 are managed services, reducing operating-system and server maintenance. The team remains responsible for code,

configuration, permissions, data handling and testing. The backend is cloud-hosted, while the prototype frontend still runs locally on each user's computer, it has not been deployed to cloud hosting.

# 3. Cloud Service/Deployment Choices and Trade-offs

## 3.1 Chosen Approach

Lambda, API Gateway and DynamoDB were selected for the small, intermittent demonstration workload. Lambda avoids an always-running server, DynamoDB transactions support coordinated booking/reservation updates and S3 stores prepared historical analytics outputs for the dashboard.

## 3.2 Alternatives Considered

Two alternatives to the serverless design were considered. The first was Amazon EC2, a virtual machine that would run the application on a server the team manages. It was not selected because the team would have to patch, secure and keep the server running even when nobody is booking a room, which is poor value for a small and intermittent workload. The second was a managed container service such as Amazon ECS. It was not selected because it needs container images, a registry and cluster configuration, which adds setup time without improving the booking workflow. AWS Lambda was chosen because its request driven model suits the prototype workload and reduces infrastructure management. The trade-offs of this choice are less control over the runtime, possible cold starts and greater dependence on AWS specific configuration.

## 3.3 Key Assumptions and Constraints

The prototype assumes a small number of concurrent demonstration users, uses fictional live rooms and test accounts and separates historical Kaggle room identifiers from demonstration rooms. It is deployed in us-east-1 within AWS Academy, with no multi-region recovery, physical attendance verification or real campus booking integration.

## 3.4 Cost-control Measures

The design avoids an always-running application server and prepares analytics outputs in advance. Costs can arise from Lambda, API Gateway, DynamoDB, S3 and CloudWatch, no production monthly cost has been established. Recommended controls include suitable log retention, removal of obsolete S3 versions and monitoring of resource consumption.

# 4. Implementation, Data Design and Security Controls

## 4.1 Application

The frontend is a static single-page site. Students can search rooms by date and group size, book one-hour slots with participants, view and cancel bookings, and check in. Occupied slots are disabled. Administrators can add and edit rooms and view reservation analytics. API calls use one configurable base address, allowing local and deployed endpoint switching. Interface testing (evidence/test-auth-ui.md and evidence/test-ui.md, 10 and 11 October 2026) found that sign-in worked for a student and an administrator and that 23 of 26 interface checks passed. Find a room has no loading message and an empty search shows only "0 rooms". The session-expiry check covered only a simulated local expiry and not a server rejection of the login, and the analytics-failure message was not tested.

The backend in src/backend/ contains the booking logic and validation and is separated from HTTP server startup so the same logic can run inside Lambda.

## 4.2 Booking Validation Rules

- Participant validation: selected participants must exist and be active, duplicates are rejected, the organiser is automatically included, administrator accounts cannot be booking participants.
- Room validation: the room must exist and be active and participant count cannot exceed capacity.
- Conflict checks: a room cannot be booked twice for the same slot and a participant cannot be in two different rooms at the same time.
- Organiser limit: an organiser may have at most two future bookings, a third future booking is rejected.
- Transactional booking: booking, room reservation and participant reservations are saved together. DynamoDB transactions and a revision guard help prevent conflicting updates.
- Mock DynamoDB tests cover validation and transaction construction. Deployed AWS tests have verified booking creation/persistence, cancellation/slot release and unauthorised operations, cloud conflict/concurrency checks are successful.

## 4.3 Cancellation

Only the organiser or authorised administrator may cancel. Cancellation retains the booking with status cancelled and releases the room and participant reservations in a transaction, allowing the slot to be reused while preserving history. The intended flow is: cancellation request → authorisation check → mark cancelled → release room → release participants. Unauthorised cancellation is tested in the backend/security evidence.

## 4.4 Data Model

DynamoDB uses four main tables: Rooms (room ID, capacity, active status), RoomlyUsers (identity, role and active status), RoomlyBookings (organiser, room, date/time and status) and RoomlyReservations (room/participant reservation records used for conflict prevention). Partition/sort keys organise records for required lookups. The overall relationship is RoomlyUsers → RoomlyBookings → RoomlyReservations ← Rooms.

Booking-state revision guarding helps prevent older updates overwriting newer states. A room-number sequence prevents duplicate room numbers during concurrent room creation. DynamoDB transactions coordinate related booking and cancellation updates so partial saves are avoided.

### 4.4.1 Attendance and Check-in

Authorised users associated with a booking may check in only within the allowed period, early or late attempts are rejected and valid check-ins record attendance. Check-in is self-reported and does not physically verify presence. No-show handling relies on stored booking/attendance information.

## 4.5 Security Controls

Cognito managed login uses the authorization-code flow with PKCE and sign-in attempt validation, API Gateway's JWT authorizer validates the Cognito issuer and audience. Authentication is separate from application authorisation: the backend maps the verified identity to RoomlyUsers and checks permissions, with administrator operations requiring the administrator flag and cancellation limited to the organiser or authorised administrator.

CORS permits the configured local frontend origins and required methods/headers but does not replace authentication or authorisation. The analytics bucket blocks public access and Lambda uses an execution role so browsers do not need AWS credentials. The AWS Academy LabRole may be broader than least privilege, production should use narrower roles.
Passwords, AWS keys and authentication tokens must not be included in source or evidence, screenshots should be redacted where necessary.

# 5. Analytics or AI/ML Feature: Data, Method, Evaluation and Limitations

## 5.1 Dataset and Provenance

The team used the University Library Room Reservations dataset published on Kaggle [1], whose listed licence is MIT and was verified on 11 October 2026. It contains historical 2024 reservations and is used only for demand analytics, not live Roomly bookings. The raw file has 51,826 rows. data/prepare_reservations.py parses timestamps, calculates duration,

applies documented exclusions and preserves room IDs as text, analytics/reproduce.py coordinates the raw-to-output workflow.

Quality checks found no invalid timestamps, missing room identifiers or exact duplicate rows. One reservation under 5 minutes and 506 over 240 minutes were excluded, producing 51,319 records and 46 room identifiers. These are analysis cleaning rules, not live booking rules. Field definitions are in data/DATA_DICTIONARY.md and provenance/licence details in data/README.md, the source timezone is unknown.

## 5.2 Method

The descriptive analysis in analytics/reservation_analysis.ipynb calculates reservation counts by room, weekday and start hour, reserved hours by room and average/median duration. It also checks overlapping reservations and dates without reservation starts. The workflow is reproducible through analytics/requirements.txt and python analytics/reproduce.py. No no-show prediction or demand-forecasting model is included.

## 5.3 Evaluation

Validation on 11 October 2026 confirmed 51,319 cleaned records after 507 exclusions, room, weekday and start-hour groupings each sum to 51,319 and the first five durations total 630 minutes (10.5 hours). Independent checks confirmed Room 105 = 2,107 reservations, Wednesday = 9,233 and 14:00 = 4,755. A separate macOS environment reproduced the raw-to-output workflow successfully with exit status 0 and “PASS: analytics reproduced successfully”, the output is in evidence/analytics-reproduction-2026-10-11.txt and validation in evidence/test-data-ai.md.

The successful reproduction demonstrates the declared workflow in a separate environment but not compatibility with every computer or AWS availability. Detailed dashboard display checks remained pending because a later GET /session returned HTTP 500, a successful administrator analytics API request confirmed API access and data delivery.

## 5.4 Results and Outputs

The notebook exports reservations_by_room.csv, reservations_by_weekday.csv, reservations_by_start_hour.csv and summary_metrics.json. Results: 51,319 reservations, 46 room identifiers, 81,622.72 reserved hours, 95.43-minute average, 90-minute median, Room 105 highest count (2,107), Room 104 highest reserved time (3,284.5 hours), Wednesday busiest weekday (9,233), 14:00 busiest start hour (4,755). These indicate periods where availability, staffing or maintenance could be planned, but represent reservation demand rather than confirmed occupancy.

# 5.5 Limitations

Reservations do not prove attendance/occupancy and the source timezone is unspecified. Two overlapping Room 234 reservations were retained because their correctness could not be determined. Nine weekend dates had no reservation starts, with no evidence of whether this represented closures, zero demand or missing data. The 507 exclusions mean the cleaned set does not represent every raw record. Historical rooms are separate from fictional

live rooms and the dataset has no capacities, identities or participant information and covers only one year.

### 5.6 Responsible Use

The analysed fields contain reservation times and room identifiers rather than personal identifiers. Results are aggregated and kept separate from fictional live rooms. Analytics does not make automated booking, access or allocation decisions, assumptions and cleaning limitations are documented so reservation demand is not mistaken for occupancy or prediction.

# 6. Testing, Scalability/Resilience and Monitoring Results

## 6.1 Test Summary

Deployed security testing on 9–11 October 2026 showed: student GET /admin/rooms = HTTP 403, no token = HTTP 401, administrator = HTTP 200, student room create/update and unauthorised booking cancellation also returned HTTP 403. An administrator-created room without an ID was assigned R006 and persisted. After the analytics permission update, student GET /analytics returned 403 and administrator GET /analytics returned 200 with historical JSON. Evidence is in evidence/test-security.md and deployment screenshots.

The historical analytics integration returned HTTP 200 and displayed 51,319 reservations, 81,622.72 reserved hours and 95.43-minute average duration, matching the exported summary. The room-list security test does not establish protection of every administrative endpoint.

### 6.1.1 Backend Test Summary

The main backend tests cover the following areas:

| **Test Area** | **Expected Behavior** |
|---|---|
| Room conflict | When 2 requests target the same room and time slot, only 1 booking should succeed |
| Participant conflict | A participant cannot be booked into 2 different rooms at the same time |
| Room capacity | A booking that exceeds the room capacity should be rejected |
| Organiser booking limit | A third future booking by the same organiser should be rejected |
| Cancellation | Cancelling a booking should release both the room and participant reservations |
| Cancellation authorisation | Another user should not be able to cancel someone else’s |

| | |
|---|---|
| | booking |
| Check-in | Check-in requests before, during and after the allowed window should return the appropriate result |
| Admin participant rule | An admin should not be allowed to become a booking participant |
| Room-number generation | Concurrent room creation should not result in duplicate room numbers |

Backend tests cover room conflict, participant conflict, capacity, the two-future-booking limit, cancellation and reservation release, cancellation authorisation, check-in timing, administrator participant restriction and concurrent room-number generation. Mock DynamoDB tests and commands are recorded in evidence/test-backend.md. Deployed AWS tests have covered booking persistence, cancellation/slot release and unauthorised operations, same-room concurrency, same-participant concurrency and no-partial-record checks are successful.

## 6.2 Scalability and Resilience Mechanism

Managed AWS services provide a foundation for handling variable request demand, but the prototype has not demonstrated campus-scale throughput. The booking backend uses transactional reservation checks and a shared revision guard to preserve consistency. Its scan-based checks and shared guard may constrain throughput as the dataset and concurrency increase.

S3 Versioning provides an implemented recovery mechanism for overwritten analytics objects. On 9 October 2026, the team uploaded two versions of a test object and retrieved both the earlier and current contents successfully. The expected contents, “version one” and “version two”, matched the downloaded versions.

This demonstrates earlier-version retrieval after an overwrite. It does not demonstrate complete application recovery or recovery of DynamoDB records. The procedure and evidence are recorded in evidence/test-resilience.md.

From the backend perspective, the current design is suitable for the prototype’s expected workload, but it has not been tested under high levels of concurrent traffic. If the system were expanded for a larger campus deployment, scan-based operations could be replaced with targeted queries and indexes and the shared revision guard would need review if contention became a problem.

# 6.3 Monitoring

CloudWatch was used to inspect Lambda execution reports and API Gateway access logs. A room-list Lambda invocation on 24 September 2026 recorded 36.53 ms execution, 37 ms billed duration and 92 MB maximum memory against 128 MB. On 10 October, the API Gateway log group was filtered for GET /session, a 15:21:24.414 UTC event returned HTTP 200 with no integration/error fields. A separate status=500 filter found no matches in the selected one-hour window. These checks verify logging and investigation capability, not

sustained performance or availability. Procedures and evidence are in evidence/monitoring.md.

### 6.4 Known Limitations

The frontend requires local setup and the cloud demonstration depends on AWS Academy. The deployment is single-region with no demonstrated whole-application recovery or high-load benchmark and IAM permissions are constrained by the laboratory role. Historical analytics describe reservations rather than occupancy or predictions, while check-in records a user action rather than physical presence. Local SQLite tests and deployed DynamoDB evidence must also be distinguished.

## 7. Cost, Sustainability and Operational Considerations

Roomly uses managed, request-driven services to reduce the need for an idle application server, while prepared analytics outputs avoid repeated analysis during dashboard requests. This design alone does not establish a measured sustainability benefit.

As an indicative workload, the team assumed about 20,000 API requests per month from demonstration users. The room list Lambda invocation recorded on 24 September 2026 was billed for 37 ms at 128 MB. At that rate, 20,000 requests would use about 92.5 GB seconds of compute, which is far below the Lambda monthly free tier of 1 million requests and 400,000 GB seconds. Lambda charges are therefore expected to be negligible. API Gateway, DynamoDB, S3 and CloudWatch Logs are billed separately and were not measured in the AWS Academy lab account. Their cost should be estimated with the AWS Pricing Calculator before any wider deployment. Suitable cost controls include setting log retention, removing obsolete S3 versions and monitoring resource use.

Operational cost depends on Lambda execution, API Gateway requests, DynamoDB reads/writes, S3 objects/versions and log storage. Before wider deployment, the team should estimate representative workload and cost using the actual configuration.

Operational maintenance includes checking failed API requests, reviewing execution logs, keeping deployed code aligned with the repository, maintaining test accounts and verifying configuration after updates. S3 versioning supports analytics-object recovery, broader recovery would require additional procedures for application configuration and database records. The submission includes source, configuration documentation and dated evidence so assessment does not depend only on continued AWS Academy access.

# 8. Team Contribution, Ethical Considerations and Reflection

## 8.1 Contribution Matrix

| Member | Role | Artefacts/ Commits | Test/ Evidence Ownership |
|---|---|---|---|
| Sharlene | Frontend | Frontend, administrator interface and dashboard; UI and authentication test records | Functional UI and authentication UI tests |
| Stanberley | Backend | Booking services, validation and DynamoDB improvements; architecture diagram; backend test dependencies | Automated backend tests and booking conflict test records |
| Shermaine | AWS & Integration | src/infrastructure/*; AWS integration, Lambda handlers, Cognito integration and analytics permission update; infrastructure documentation and project manifest | Deployed functional and security tests, S3 version retrieval and CloudWatch monitoring |
| Yuchen | Data & Analytics | Dataset cleaning and preparation, data dictionary, analysis notebook, reproduction scripts and exported analytics outputs | Dataset and analytics validation records |

## 8.2 Ethical Considerations

The live prototype uses fictional rooms and test accounts. Historical analytics use a separate public Kaggle reservation dataset containing room identifiers and timestamps, fictional live rooms, locations and capacities are labelled as demonstration values. AI-assisted code and open-source baselines are declared with licences and team verification in AI_USE_DECLARATION.md.

## 8.3 Individual Reflections

**Shermaine**

I mainly handled AWS deployment and integration across the frontend, API Gateway, Lambda, DynamoDB, Cognito and analytics service and collected cloud/security evidence. This showed me that a working interface is not enough: handlers, routes, authorizers and callback URLs must agree. I also learnt to separate authentication from authorisation through student, administrator and unauthenticated tests. Some deployment settings/tests were documented late, next time I would keep configuration records and repeatable tests alongside each deployment change and use narrower roles where possible.

**Stanberley**

I was mainly responsible for the backend booking system: users, rooms, bookings and participants, participant validation, duplicates, organiser inclusion, capacity, booking limits, room/participant conflicts and cancellation release. I also added tests for room conflicts, participant conflicts, capacity, cancellation, authorisation, check-in timing, admin participants and room-number generation and worked on DynamoDB transactions, reservation records, repeated requests and the revision guard. The project taught me that backend reliability means thinking about failure cases and integration, especially when something works locally but the connected AWS system can still have issues.

**Sharlene**

I was responsible for the frontend and user experience, including room selection, booking, participant selection and analytics screens and agreed request/response formats with the backend. I worked on the booking flow, My Bookings, cancellation, admin visibility and API handling and performed UI/authentication testing. The project taught me to think beyond appearance and consider failure cases, debugging and testing.

**Yuchen**

I handled data and analytics by cleaning the historical dataset, documenting 507 removed records, updating the data dictionary/source information, maintaining the analysis notebook, generating four dashboard outputs and checking reservation counts, reserved hours and duration calculations. I learnt that reproducible analysis depends on documented cleaning, file locations and software and that room IDs must be loaded as text to preserve leading zeros such as 033. A separate Python-environment reproduction test confirmed the workflow. A limitation is that reservations do not prove occupancy, future work could add check-in/occupancy data and, if enough history becomes available, forecasting.

# References

[1] Aceeedev, "University Library Room Reservations," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/aceeedev/university-library-room-reservations

# Appendices

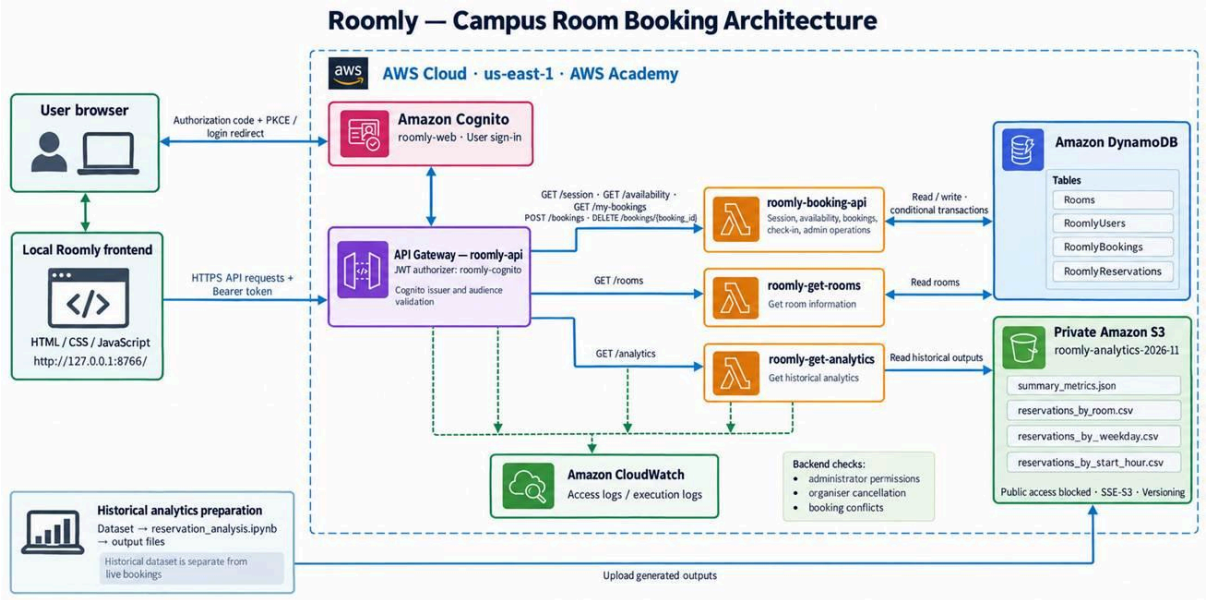



*Figure 1. Roomly architecture. The browser does not access DynamoDB or S3 directly, API Gateway and Lambda mediate requests, while Lambda execution roles and backend checks enforce access.*

## Appendix A: Evidence Index

| Claim | Evidence file |
|---|---|
| Booking persists, cancellation releases slot | evidence/test-functional.md |
| Security controls (401/403/200) | evidence/test-security.md |
| Analytics reproduced and validated | evidence/test-data-ai.md, evidence/analytics-reproduction-2026-10-11.txt |
| Resilience (S3 versioning) | evidence/test-resilience.md |
| Monitoring | evidence/monitoring.md |
| Interface and sign-in tests | evidence/test-ui.md, evidence/test-auth-ui.md |
| Architecture diagram | evidence/architecture.png |